

IBKR Paper-Trading Project Engineering Manual / 系统工程手册

Manual version: v0.1.0 | Release: Foundation Baseline / 初始架构基线版

Status: Draft / Not yet fully system-validated

Generated: 2026-07-14T00:01:49.328134+00:00 | Git branch: feature/android-integration-mvp | Git commit: 345fe379a8b06d1f7e1e1432edda1d78db54cb5f

Commit link:

<https://github.com/liuhaitian888-wq/openclaw-ibkr-paper-trading/commit/345fe379a8b06d1f7e1e1432edda1d78db54cb5f>

This Foundation Baseline is a draft system architecture manual. It is repository-backed, but it is not a claim that every runtime path has been fully validated in production-like paper sessions.

Table of Contents

Table of Contents	2
Project Introduction	3
How to Read This Manual	3
5-minute overview	3
30-minute developer overview	3
Deep maintenance reference	3
System Overview	3
Complete System Architecture	4
Runtime Lifecycle	5
Data Flow, Signal Flow, Control Flow, and Error Flow	5
Module Communication Contract	6
Six-Layer Stock Pool	7
IBKR/TWS Integration	8
Risk and Execution Architecture	8
Database Architecture	9
Log and Observability Architecture	11
Autonomous Review Framework	12
Failure and Recovery Scenarios	13
Developer Operations	14
Source-Code Reference	14
Appendix	18
Glossary	18
Configuration Reference	19
Facts Requiring Manual Confirmation	19
Data Inventory	19
Git/Version Information	19

Project Introduction

- The project is an IBKR paper-trading automation and evidence system. It combines candidate discovery, market data checks, strategy decisions, risk gates, paper execution adapters, SQLite ledgers, reports, and logs.
- The safety boundary is paper-only. Live trading is intentionally outside the validated system boundary and must not be inferred from paper readiness.
- The manual is ordered System → Layers → Flows → Modules → Storage → Operations → Code Reference so a new developer can reason about architecture before reading implementation details.

Metadata	Value
Manual version	v0.1.0
Release	Foundation Baseline / 初始架构基线版
Validation status	Draft / Not yet fully system-validated
Code Git commit	345fe379a8b06d1f7e1e1432edda1d78db54cb5f
Git branch	feature/android-integration-mvp
Generation date UTC	2026-07-14T00:01:49.328134+00:00

How to Read This Manual

5-minute overview

- Read System Overview, Layered Architecture, and the Mode 9 lifecycle diagram.
- Remember the invariant: candidate modules may produce facts or intents, but broker interaction must pass risk and execution gates.

30-minute developer overview

- Read flows separately: data flow, signal flow, control flow, and error flow use different arrows and evidence.
- Then read the module contract and six-layer pool sections before touching code.

Deep maintenance reference

- Use Database Architecture, Observability, Failure/Recovery, Developer Operations, and Source-Code Reference.
- Every review claim should cite runtime evidence: table row, report file, timestamp, cycle_id, order_id, or log entry.

System Overview

Complete system mind-map / architecture infographic

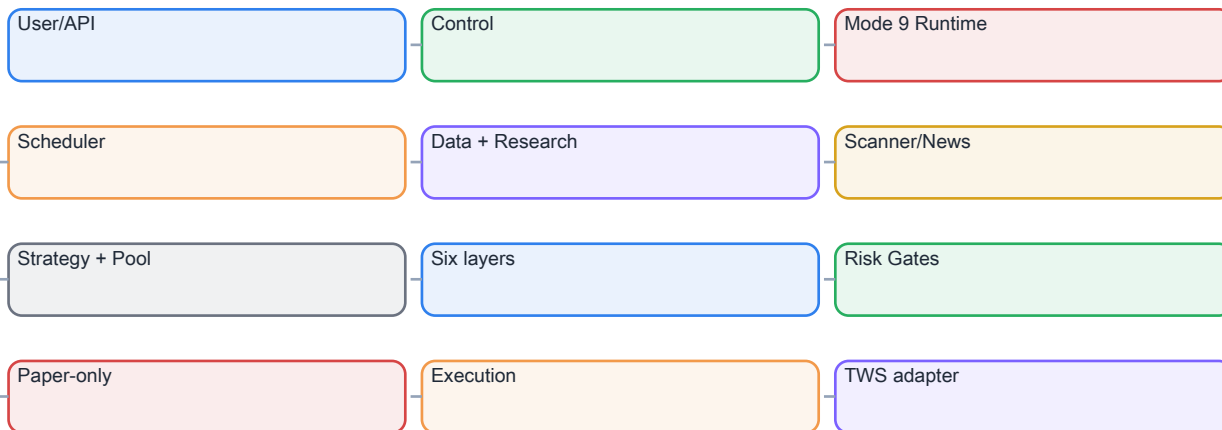


Diagram: Complete system mind-map / architecture infographic. Source file: docs/manual/diagrams/complete_system_mind_map.svg

- Operational boundary: the repository controls local Python services, reports, SQLite, and IBKR/TWS paper interaction through configured ports. It does not own the external TWS desktop process.
- Paper-only boundary: `TRADING\_MODE` accepts DRY_RUN or PAPER; live trading is not an enabled mode in `trading/config.py`.
- External systems: IBKR TWS/IB Gateway paper socket, local filesystem, SQLite, CSV universes, generated reports, and operator-controlled scripts.

Complete System Architecture

Layered system architecture diagram

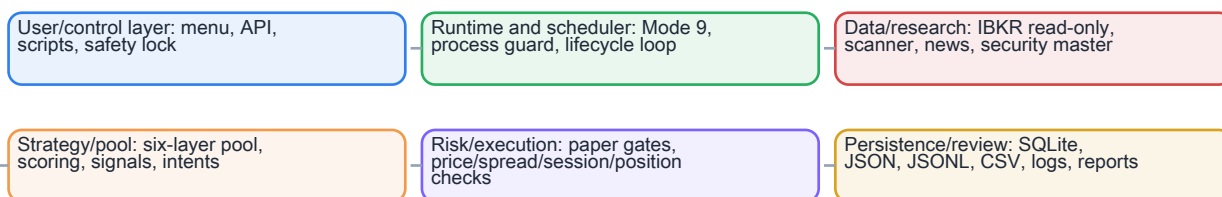


Diagram: Layered system architecture diagram. Source file: docs/manual/diagrams/layered_system_architecture.svg

Layer	Core responsibility	Representative repository paths
User/control	Menus, scripts, API requests, lock state, developer commands.	scripts/*.command, TRADING_LOCK.md, SECURITY_MODEL.md
Runtime/scheduler	Mode 9 cycles, duplicate-process lock, sleep cadence, status reports.	scripts/run_autonomous_trading_agent.py, trading/process_guard.py, trading/autonomous_runtime.py

Layer	Core responsibility	Representative repository paths
Data/research	Universe, scanner, news, quotes, account and position state.	trading/ibkr_readonly.py, trading/ibkr_streaming.py, trading/news_*.py, data/*.csv
Strategy/pool	Candidate scoring, six-layer pool, strategy signals, buy/sell intents.	trading/pool_state.py, trading/strategy*.py, trading/buy_decision_engine.py
Risk	Paper-only limits, price/spread/session/position/protection gates.	trading/risk.py, trading/session_policy.py, trading/position_protection.py
Execution	Paper order construction, transmit behavior, readback, callbacks.	trading/tws_paper.py, trading/paper_order_executor.py, trading/order_intents.py
Audit/storage	SQLite evidence, JSON/JSONL/CSV reports, logs.	trading/paper_audit_db.py, trading/sqlite_event_store.py, reports/, logs/

Runtime Lifecycle

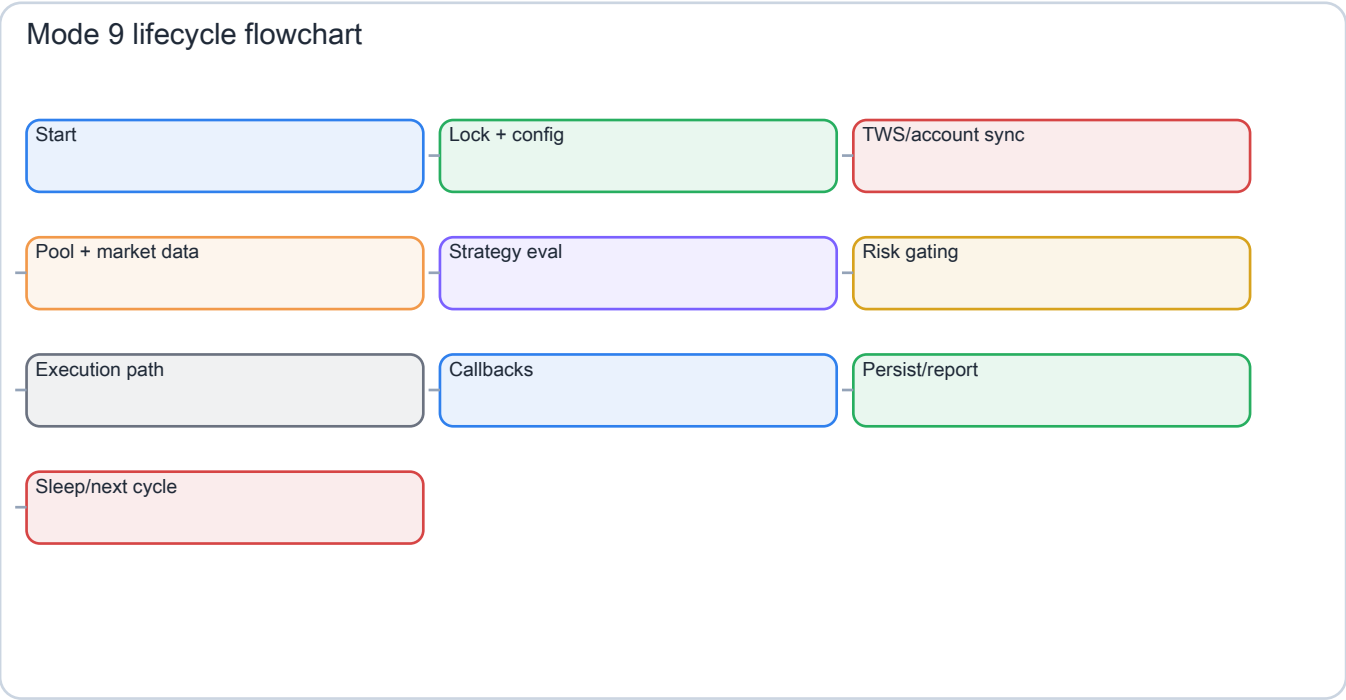


Diagram: Mode 9 lifecycle flowchart. Source file: docs/manual/diagrams/mode9_lifecycle_flowchart.svg

Stage	Repository-backed behavior
Startup	Operator/script starts the runtime; config is read from environment and local files.
Duplicate-instance lock	`trading/process_guard.py` records/validates the execution-writer owner.
Configuration validation	`Settings.load` constrains mode to DRY_RUN/PAPER and loads limits, tokens, ports, symbols, and audit DB.
IBKR connection	Read-only or paper adapters connect to configured localhost TWS paper port when invoked by runtime diagnostics or execution path.
Account/position synchronization	Account, position, PnL, open orders, and quote states are refreshed into reports/state buses.
Pool loading	Security master and pool membership evidence feed the canonical six-layer pool.
Market-data acquisition	Readonly, streaming, or report-cache data are used depending on session and capability.
Strategy evaluation	Strategy modules create signals and intents; they do not directly call IBKR.
Risk gating	Session, quote freshness, spread, price, order value, position, and protection gates must pass.
Execution	Only paper execution adapters may submit orders; report-only modules emit evidence only.
Callbacks	IBKR callbacks update buses, errors, commissions, order status, and stale/connection state.
Persistence/reporting	SQLite rows, JSON reports, JSONL events, CSVs, and logs are written.
Sleep/next cycle	Runtime sleeps according to configured cadence and repeats.
Shutdown/recovery	State is reconstructed from reports/SQLite; stale or inconsistent state blocks execution.

Data Flow, Signal Flow, Control Flow, and Error Flow

Market-data request and callback sequence diagram

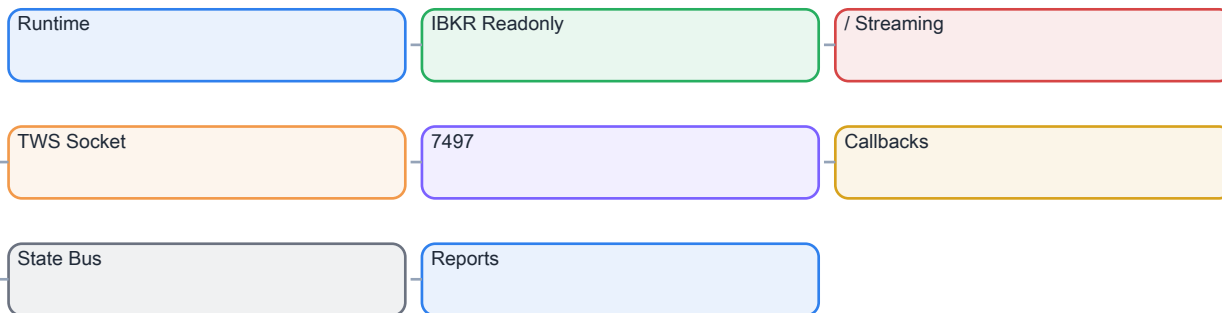


Diagram: Market-data request and callback sequence diagram. Source file: docs/manual/diagrams/market_data_sequence.svg

Trading-signal and order-execution flowchart

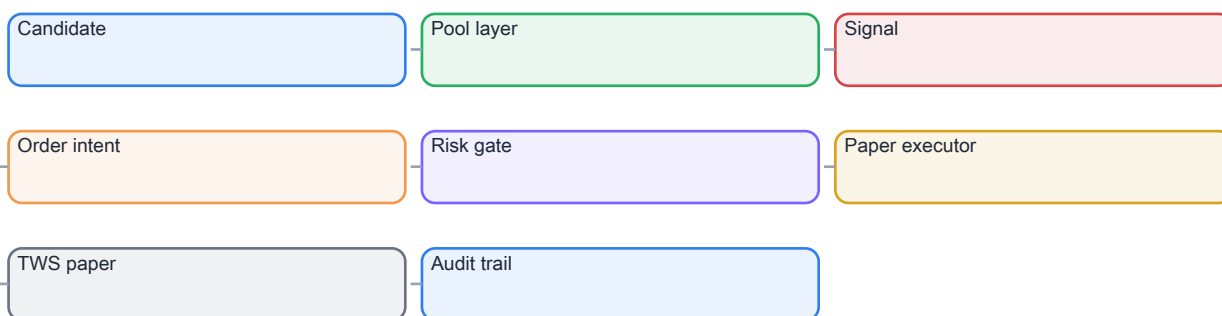


Diagram: Trading-signal and order-execution flowchart. Source file: docs/manual/diagrams/trading_signal_order_flow.svg

- Data flow is blue: external/reference data, quotes, account state, positions, news, scanner output, and CSV universes move into buses, reports, and SQLite.
- Signal flow is green: candidate scoring and strategy modules promote symbols through pool layers, then emit `strategy_signals` or `order_intent_events`.
- Control flow is orange: operator commands, API endpoints, lock/token state, and transmit flags decide whether a request may advance toward TWS.
- Error flow is red/purple: exceptions, stale states, callback errors, rejections, and blocked reasons are persisted and should stop execution before review.

Module Communication Contract

Module communication/dependency diagram

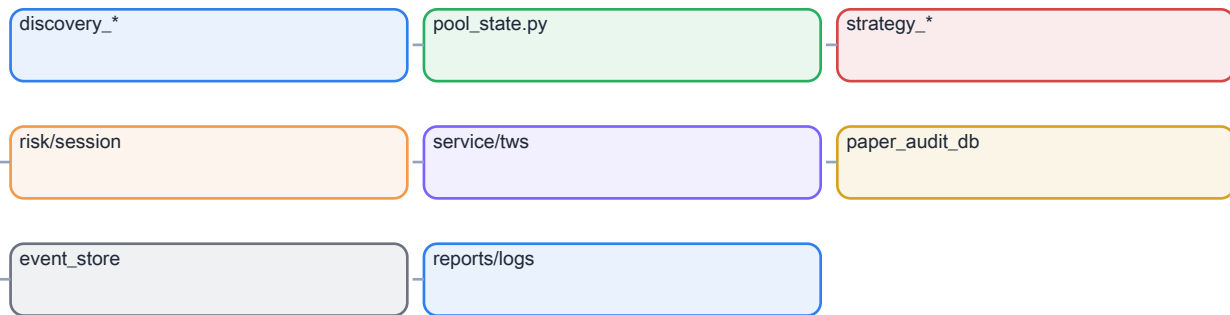


Diagram: Module communication/dependency diagram. Source file: docs/manual/diagrams/module_dependency_diagram.svg

Module/group	Responsibility	Inputs	Outputs	Forbidden direct dependency
Discovery/scanner/news	Find and structure candidate evidence.	CSV universes, scanner/news providers, reports.	discovery_candidates, raw/structured news, source status.	Must not submit broker orders.
Pool state/manager	Own canonical six-layer stock state and transitions.	Discovery, eligibility, quotes, signals, order callbacks.	canonical_pool_state, transition events, pool reports.	Must not bypass risk/execution gates.
Strategy modules	Evaluate positions/candidates and emit decisions.	Pool membership, quotes, bars, account state.	strategy_signals, buy/sell decisions, intents.	Must not call TWS directly.
Risk/session/position gates	Approve, block, or require review.	Intent, session, quote, account, position, config.	blocked_reason, readiness reports, risk events.	Must not mutate broker state except via approved execution path.
Execution adapters	Construct paper orders and handle TWS acknowledgements.	Approved proposal/intents, settings, TWS client.	order_requests, order status, readback evidence.	Must not enable live mode.
Audit/reporting	Persist runtime evidence and review artifacts.	Events, reports, callbacks, decisions.	SQLite, JSON, JSONL, CSV, logs.	Must not make trading decisions by itself.

Six-Layer Stock Pool

Six-layer stock-pool diagram

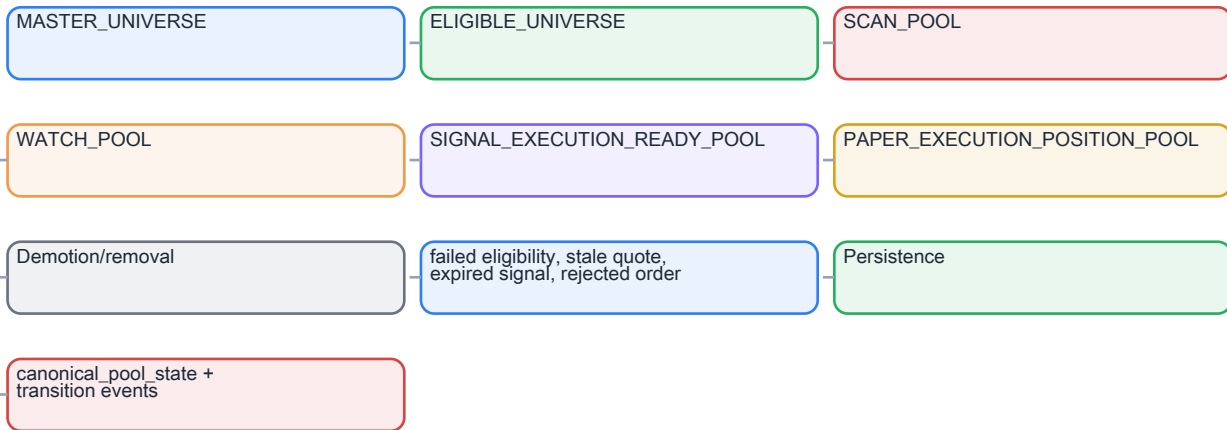


Diagram: Six-layer stock-pool diagram. Source file: docs/manual/diagrams/six_layer_stock_pool.svg

Layer	Purpose	Promotion	Removal/demotion	Capacity/refresh
MASTER_UNIVERSE	Global security master.	Symbol discovered or manually seeded.	Disabled, unsupported type/currency.	Capacity is file/report/db bounded; refreshed by security master and pool builders.
ELIGIBLE_UNIVERSE	Hard eligibility passed.	Contract resolved, market data available, price/liquidity checks pass.	Eligibility failure or explicit block.	Refreshed by eligibility evaluation.
SCAN_POOL	Coarse strategy/scanner nomination.	Strategy/source nominates with score and TTL.	TTL expiry or lower-quality evidence.	TTL configured by call site; records in transition events.
WATCH_POOL	Quote-ready active monitoring.	Fresh quote and spread check pass.	Stale quote, spread failure, TTL expiry.	Refresh follows monitoring and quote freshness policy.
SIGNAL_EXECUTION_READY_POOL	Signal is execution-ready but not yet a position.	Signal passes score, event/risk readiness, and duplicate-order checks.	Expired signal, active order conflict, risk failure.	Bound by strategy and order gating.
PAPER_EXECUTION_POSITION_POOL	Paper position/order lifecycle under monitoring.	Order accepted/filled or position open.	Closed, rejected, expired, or reconciled out.	Refreshed by callbacks, open orders, positions, and reports.

IBKR/TWS Integration

- Python entry points use IBKR adapters such as ``trading/tws_paper.py``, ``trading/ibkr_readonly.py``, ``trading/ibkr_streaming.py``, and callback bridge/audit modules.
- Socket/API connection defaults are loaded from environment: TWS host ``127.0.0.1``, paper port ``7497``, and client IDs from ``trading/config.py``.
- Request path: service validates proposal, reserves audit idempotency key, constructs order, submits to paper adapter only after gates pass.
- Callback path: TWS callbacks update order status, errors, account/position/quote buses, and generated reports.
- Reconnection and reconciliation depend on stale-state detection, readback, account/open-order refresh, and SQLite/report evidence.

Risk and Execution Architecture

- Paper-only restrictions: ``Settings.load`` accepts only DRY_RUN or PAPER. Live order submission is intentionally absent from the validated mode set.
- BUY/live/options gates: reports show paper-only and report-only flags; options execution forbids live options and naked/market option strategies.

- Price/spread/session validation: price normalizer, session policy, quote readiness, spread checks, market session reports, and market quote crosscheck all contribute blockers.
- Position protection: protective sell, gap escape, profit lock/take profit/trailing profit, and lot protection are tracked separately from new BUY logic.
- Order construction and transmit behavior live in `trading/tws\_paper.py`; staged orders use `transmit=False`, paper submission uses explicit transmit behavior after gates.
- Rejection handling writes blocked reasons, IBKR errors, readback status, and audit rows so review can reconstruct the failed path.

Database Architecture

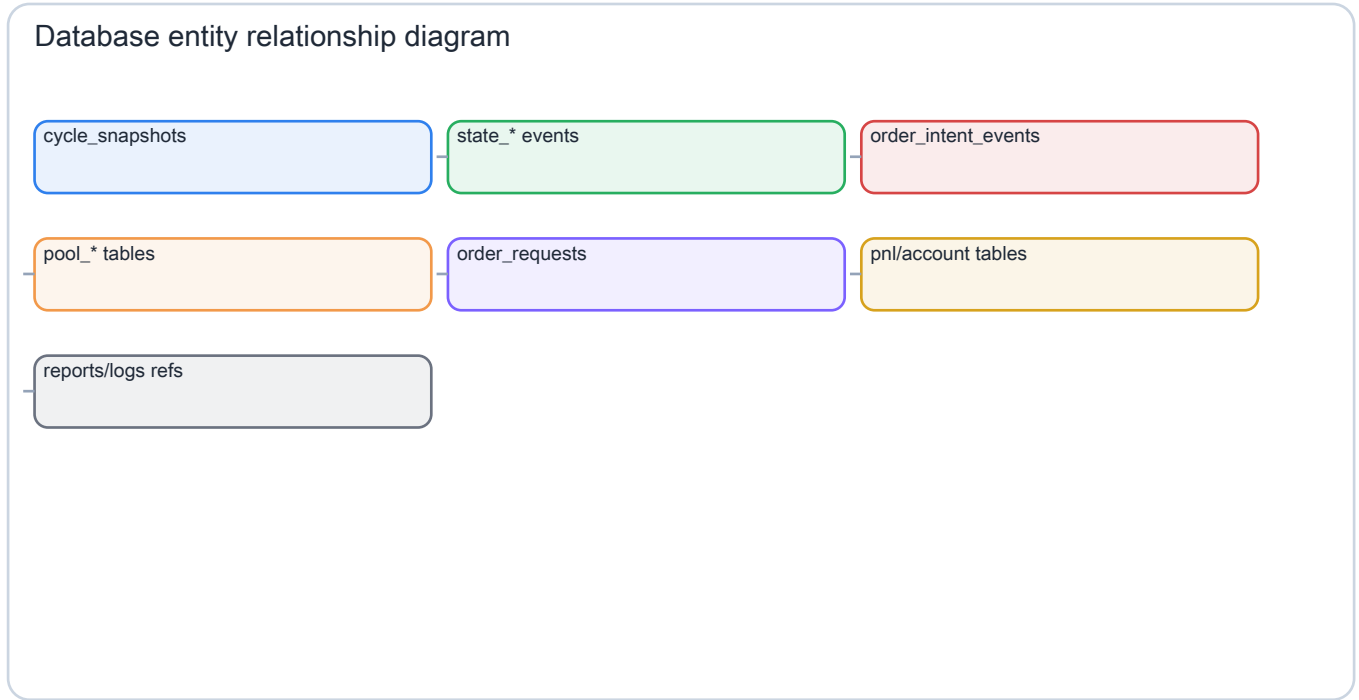


Diagram: Database entity relationship diagram. Source file: docs/manual/diagrams/database_er_diagram.svg

SQLite database inspected: /Users/nbhsbgmb/Documents/New project/trading_audit.sqlite3. Table count at generation: 83.

Table	Rows	Primary key	Purpose	Writer / readers
account_pnl_snapshots	3	trading_date/timestamp/symbol depending on table	PnL snapshots, summaries, and attribution evidence.	account/PnL report builders / risk review, performance review, account dashboard
account_snapshots	21	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
account_state_events	583	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
account_state_snapshots	17	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
bootstrap_state_events	192	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
buy_decisions	1143	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
callback_wiring_events	428	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
candidate_score_components	41910	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
candidate_scores	4191	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
canonical_pool_state	66	symbol or transition/event id when present	Canonical stock-pool membership, transitions, and layer state.	trading/pool_state.py, trading/pool_manager.py, auto-open pipeline / pool reports, strategy selection, autonomous review

Table	Rows	Primary key	Purpose	Writer / readers
canonical_pool_transition_events	-1	symbol or transition/event id when present	Canonical stock-pool membership, transitions, and layer state.	trading/pool_state.py, trading/pool_manager.py, auto-open pipeline / pool reports, strategy selection, autonomous review
commission_events	0	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
cycle_snapshots	241	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
daily_pnl	21	trading_date/timestamp/symbol depending on table	PnL snapshots, summaries, and attribution evidence.	account/PnL report builders / risk review, performance review, account dashboard
daily_pnl_summary	3	trading_date/timestamp/symbol depending on table	PnL snapshots, summaries, and attribution evidence.	account/PnL report builders / risk review, performance review, account dashboard
discovery_candidates	8205	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
discovery_runs	127	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
discovery_source_status	1651	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
event_risk_events	634	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
event_risk_sell_events	18	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
event_store	444	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
execution_ledger	0	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
execution_state_events	211	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
fast_order_guidance_events	1524	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
full_paper_run_cycles	7	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
gap_escape_events	144	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
gap_risk_events	630	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
ibkr_error_events	162	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
ibkr_news_interface_tests	762	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
ibkr_news_provider_status	762	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
ibkr_scanner_results	3048	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
ibkr_scanner_runs	127	timestamp/discovery_run_id/symbol	Discovery and candidate-selection evidence.	discovery, scanner, news, candidate scoring modules / pool manager, strategy review, AI review pipeline
lot_ledger	126	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
market_data_snapshots	126	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
market_quote_crosscheck_events	42	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
market_session_events	6	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
news_candidate_events	508	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics

Table	Rows	Primary key	Purpose	Writer / readers
open_order_state_events	211	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
options_hedge_plans	126	not declared as SQLite PRIMARY KEY unless noted by schema	Runtime evidence table.	module-specific writer / diagnostic reports and review tools
options_order_events	8	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
order_conflict_events	16	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics
order_intent_events	1784	event_id when present; otherwise append-only rowid	Append-only runtime event evidence.	module-specific event writers via trading/paper_audit_db.py or SQLiteEventStore / reports, review collector, replay diagnostics

Additional table schemas and counts are preserved in `docs/manual/generated/manual\_facts.json`; diagnostic SQL is in `docs/manual/queries/database\_diagnostics.sql`.

Log and Observability Architecture

Logging and observability flowchart

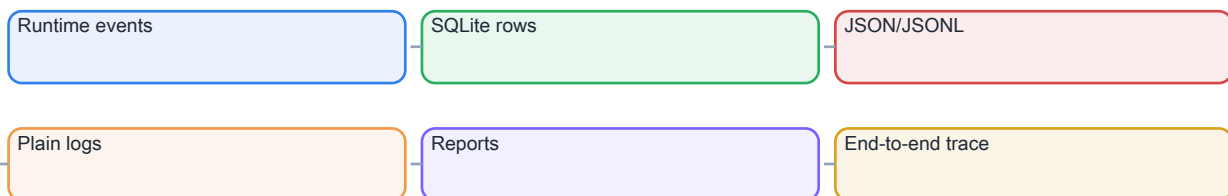


Diagram: Logging and observability flowchart. Source file: docs/manual/diagrams/logging_observability_flowchart.svg

- Log categories include API logs, autonomous agent logs, launchd logs, scheduler logs, market-data audit logs, and report-builder logs.
- Structured evidence is primarily SQLite, JSON, JSONL, and CSV; plain-text logs are used for process/system output and failures.
- Correlation fields include cycle_id, discovery_run_id, snapshot_id, intent_id, order_id, event_id, symbol, timestamps, and report_path.
- Timestamps are mostly UTC ISO strings in generated evidence; local operational docs mention Europe/Berlin where daily limits or operator context require it.

Artifact	Inspection role
reports/account/latest.json	account, buying_power, cash, currency, daily_pnl, errors, net_liquidation, open_order_count, pnl_timestamp, position_count, realized_pnl, source
reports/account_dashboard/latest.json	buy_freeze_active, buying_power, cash, gross_position_value, live_trading_enabled, mode, net_liquidation, number_of_open_orders, number_of_positions, open_orders_age_sec, paper_only, pnl_age_sec
reports/account_execution_readiness/latest.json	account_execution_ready, blocking_reasons, delayed_data_used_for_execution, live_quote_confirmed, live_quote_required, open_orders_ready, paid_snapshot_used, pnl_ready, positions_ready, protection_coverage_ready, quotes_ready, regulatory_snapshot_used
reports/account_state/latest.json	account_id, blocked_reason, buying_power, cash, force_refresh, freshness, gross_position_value, metadata, mode, net_liquidation, open_orders, orders_cancelled

Artifact	Inspection role
reports/autonomous_agent/latest.json	account_state_manager, agent_running, blocked_reason, canonical_pool_state, created_at, cycle, errors, event_risk, event_router, execution_enabled, gap_escape, gap_risk
reports/buy_sell_modularization/latest.json	buy_modules, buy_sell_use_same_cycle_snapshot_refs, discovery_run_id, fast_guidance_can_submit_orders, final_executor_enabled, news_can_submit_orders, pool_manager_can_submit_orders, scanner_can_submit_orders, sell_modules, timestamp
reports/buy_sell_separation/latest.json	answers, source, timestamp
reports/buy_sell_strategy_sync/latest.json	account_state_event_driven, buy_intent, buy_object_has_protection_plan_required, buy_object_has_protection_plan_status, buy_sell_decisions_archived_separately, buy_sell_engines_can_submit_orders_directly, buy_sell_use_same_snapshot_id, cycle_snapshot_is_decision_time_copy, fixed_30_second_cycle_used_for_execution, freshness_policy, ibkr_paper_orders_submitted, live_orders_submitted
reports/candidate_scoring/latest.json	discovery_run_id, records, timestamp
reports/cycle_snapshot/latest.json	account_state_ref, created_at, cycle_id, event_driven_snapshot, execution_state_ref, fixed_30_second_cycle_used_for_execution, immutable, market_data_ref, market_session_ref, news_state_ref, open_order_state_ref, pnl_state_ref
reports/daily_pnl/latest.json	last_insert_at, latest_rows_sample, missing_fields, remaining_gap, row_count, source, table_exists, timestamp
reports/data_bus_architecture/latest.json	buses, buses_logically_separated, decision_snapshots_reference_all_buses, logical_bus_count, timestamp
reports/discovery_orchestrator/latest.json	candidate_count, candidates, discovery_run_id, ibkr_paper_order_submission, live_trading_enabled, local_simulation_enabled, orders_submitted, source, source_status, timestamp
reports/event_risk/latest.json	config, source, status, symbols, timestamp
reports/event_router/latest.json	action_count, events, report_only, source, submitted_orders, timestamp
reports/execution_ledger_split/latest.json	blocked_reasons, buy_separated_from_sell, counts, intents, live_trading_used, market_order_used, order_submitted_count, paid_snapshot_used, ready_blocked_reasons, ready_for_next_active_session_5_minute_paper_test, regulatory_snapshot_used, source
reports/fast_order_guidance/latest.json	discovery_run_id, fast_guidance_can_submit_orders, generated_order_intent_candidates, orders_submitted, records, source, timestamp
reports/final_auto_open_discovery_news_simulation_audit/latest.json	all_news_sources_under_registry, buy_sell_use_same_cycle_snapshot_refs, discovery_enabled, discovery_run_id, discovery_universe_size, fast_guidance_generated_order_intents, hot_pool_size, ibkr_news_interfaces_failed, ibkr_news_interfaces_tested, ibkr_news_interfaces_work, ibkr_news_providers_available, ibkr_paper_orders_submitted
reports/final_full_paper_automation_audit/latest.json	answers, cycle_id, full_paper_run_report, source, sqlite_tables, timestamp
reports/final_mode9_infrastructure_audit/latest.json	answers, features, source, timestamp
reports/fixed_symbol_audit/latest.json	findings, source, timestamp
reports/force_refresh_dry_run/latest.json	blocked_reason, execution_ready_after_refresh, force_refresh_ok, force_refresh_performed, open_orders_refreshed, orders_cancelled, orders_submitted, paid_market_data_request_used, pnl_refreshed, positions_refreshed, protection_coverage_refreshed, quotes_refreshed
reports/full_paper_rollout/latest.json	failed_stages, live_orders_submitted, market_orders_submitted, options_orders_submitted, orders_cancelled, orders_submitted, passed_stages, source, stages, timestamp
reports/full_paper_run/latest.json	auto_buy_enabled, auto_buy_inside_full_paper_run, blocked_reason, cycle_id, full_automatic_buy, full_paper_autonomous_long_run, full_paper_run_enabled, gap_escape_enabled, gap_escape_execution, live_orders_submitted, live_trading_enabled, market_orders_submitted
reports/gap_escape/latest.json	config, live_enabled, paper_only, source, status, submitted_count, symbols, timestamp
reports/gap_risk/latest.json	account_net_liquidation, config, risk_events, source, status, symbols, timestamp
reports/ibkr_callback_dry_run/latest.json	bootstrap, bus_status, callback_counts, callbacks_registered, connected, duration_seconds, errors, last_event_seen_at, orders_cancelled, orders_submitted, paid_snapshot_used, regulatory_snapshot_used
reports/ibkr_callback_wiring/latest.json	account_callbacks_wired, ask_fresh, bid_fresh, blocked_reason, bus_ok, bus_running, bus_state_sources, callback_wiring, callbacks_registered, execution_callbacks_wired, expected_live_bid_ask, ibkr_connected

Autonomous Review Framework

Autonomous Review pipeline diagram

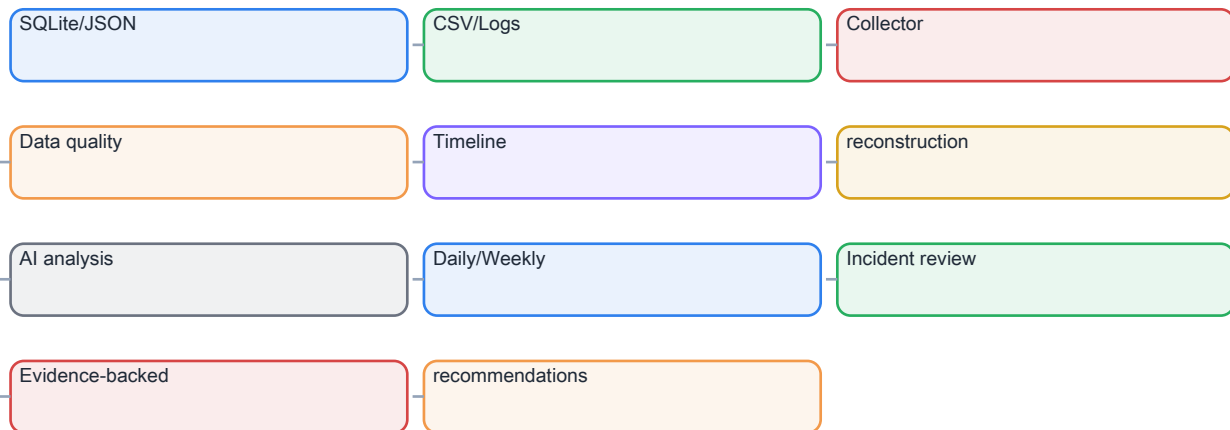


Diagram: Autonomous Review pipeline diagram. Source file: docs/manual/diagrams/autonomous_review_pipeline.svg

- The AI review pipeline must prioritize generated runtime evidence over source-code assumptions.
- Daily review: summarize cycles, blockers, stale data, submitted/blocked orders, and account/PnL changes.
- Weekly review: compare candidate quality, pool transitions, risk blocks, execution quality, and performance attribution.
- Incident review: reconstruct a timeline from SQLite rows, reports, JSONL events, logs, callbacks, and TWS readback.
- Strategy/risk/execution/performance/data-quality reviews must cite row/table/file/timestamp/cycle_id/order_id/log evidence.

Failure and Recovery Scenarios

TWS disconnect and recovery flowchart

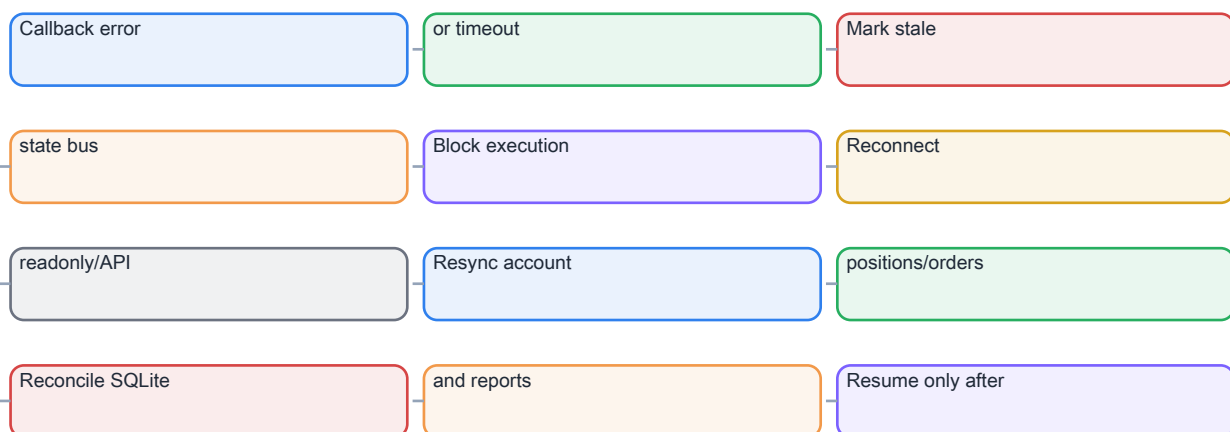


Diagram: TWS disconnect and recovery flowchart. Source file: docs/manual/diagrams/tws_disconnect_recovery.svg

Scenario	Expected propagation	Recovery evidence
TWS disconnected	Callback/socket error, stale state, execution blocked.	ibkr_error_events, position_guard/latest.json, logs.
Stale market data	Quote readiness false, spread/quote gates block execution.	quote_state_events, market_quote_crosscheck_events.

Scenario	Expected propagation	Recovery evidence
Missing bid/ask	Market session crosscheck determines whether normal closed-market condition or feed problem.	market_session/latest.json, market_quote_crosscheck/latest.json.
Market closed	Session policy disables auto trade or blocks readiness.	market_session_events, paper_order_readiness_events.
Duplicate process	Process guard reports duplicate PIDs or accepted lock owner.	reports/process_guard/latest.json, .runtime.lock.
Database locked	SQLite writer exception should fail current evidence write and be visible in logs.	plain logs plus missing/partial row checks.
Callback timeout	State remains stale and execution is blocked.	subscription_state_events, callback_wiring_events.
Rejected order	Rejected status/IBKR error written, order readback not accepted.	order_requests, ibkr_error_events, order_readback_events.
Partial execution	Execution state and order status indicate partial fill; position/PnL reconciliation required.	execution_state_events, position_pnl_snapshots.
Inconsistent account state	Freshness and reconciliation checks block execution.	account_state/latest.json, realtime_account_state_bus/latest.json.
Restart after crash	Recover from SQLite/reports/open order readback before resuming.	cycle_snapshots, open_order_state_events, process_guard.

End-to-end event replay example

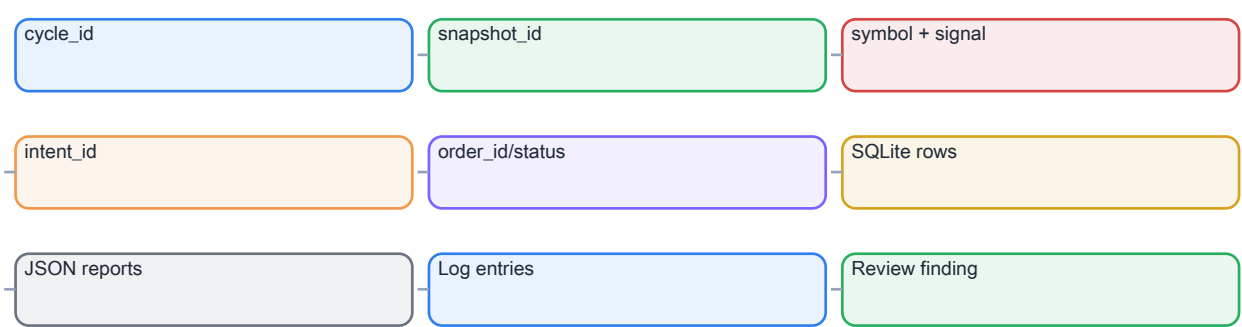


Diagram: End-to-end event replay example. Source file: docs/manual/diagrams/event_replay_example.svg

Developer Operations

- **Start/stop:** use documented command scripts and runbooks; avoid direct runtime entry unless you understand the lock/token state.
- **Confirm single runtime:** inspect ``reports/process_guard/latest.json`` and ``.runtime/execution_writer.lock``.
- **Inspect current state:** read ``reports/account_state/latest.json``, ``reports/realtime_account_state_bus/latest.json``, and ``reports/mode9_infrastructure_status/latest.json``.
- **Inspect SQLite:** run read-only ``sqlite3 trading_audit.sqlite3`` queries from ``docs/manual/queries/database_diagnostics.sql``.
- **Inspect logs:** start with ``logs/trading_api.log``, ``logs/autonomous_agent.log``, and `launched` logs.
- **Generate review:** collect SQLite rows, reports, JSONL events, CSVs, and logs; require citations for every AI conclusion.
- **Verify no live trading:** check ``TRADING_MODE``, ``live_trading_enabled`` report fields, ``TRADING_LOCK.md``, and ``SECURITY_MODEL.md``.

Source-Code Reference

This section intentionally appears after the architecture sections. It preserves the file-level view while keeping system behavior first. Paths are repository-relative and were extracted from real files.

Path	Purpose	Classes	Functions
trading/___init___py	Core types and safeguards for the trading service.		

Path	Purpose	Classes	Functions
trading/account_state_manager.py	Project file.		build_account_state, force_refresh_state, build_force_refresh_dry_run, refresh_streaming_quote_diagnostics, overlay_streaming_quotes, symbols_from_guard, require_fresh_state_for_execution, build_quote_states, quote_state_from_guard, quote_execution_readiness
trading/audit.py	Project file.	AuditLog	_notional
trading/audit_writer.py	Single-writer audit queue for SQLite append-only ledgers.	AuditEvent, AuditWriter	
trading/auto_open_pipeline.py	Auto-open local discovery, news, dynamic pool, and simulation pipeline. This module is deliberately local/simulated. It can create candidates, guidance, order intents, and SIMULATE	PipelineConfig	run_auto_open_pipeline, assert_simulation_safety, run_discovery_orchestrator, run_ibkr_scanner, run_news_pipeline, news_provider_status, ibkr_news_interface_tests, raw_news_fixture, structure_news_event, run_dynamic_pool
trading/autonomous_runtime.py	Project file.	Scanner, AutonomousRuntimeConfig, RuntimeCycleReport, AutonomousPaperRuntime	build_runtime, _read_open_orders, _report, _health_ready, _proposal_from_event, _quote_from_event, _float_or_none, _bool_env
trading/buy_decision_engine.py	BUY decision engine. Emits BUY intents only; never calls IBKR directly.		build_buy_intent, can_submit_order, submit_order
trading/candidate_research.py	Project file.	EvidenceItem, LlmPreReview, CandidateDraft, HardAuditConfig, HardAuditResult, CandidateReport	build_candidate_report, audit_candidate, normalize_draft, load_candidate_draft, candidate_draft_from_dict, write_candidate_report, candidate_report_from_dict, load_candidate_report, _profile_from_dict, _pre_review_from_dict
trading/candidate_scoring.py	Candidate scoring helpers.		score
trading/config.py	Project file.	Settings	_read_bool, _load_or_create_api_key, _load_optional_file_value, _load_allowed_symbols, _symbols_from_file, _read_symbol_tuple, _read_optional_float
trading/cycle_snapshot.py	Immutable decision-time snapshots over the realtime account state bus.	CycleSnapshot	create_cycle_snapshot, write_report
trading/dashboard.py	Project file.		render_dashboard_html
trading/data_buses.py	Logical data bus architecture for realtime decisions.	MarketDataBus, NewsTriggerBus, DiscoveryPoolBus	build_data_bus_architecture_report
trading/discovery_orchestrator.py	Unified discovery orchestrator endpoint.		run
trading/event_risk_control.py	Project file.	EventRiskConfig	load_event_risk_config, build_event_risk_report, recommended_event_action, load_local_events, write_report, _event_for_symbol, _latest_net_liquidation, _bool_env, _float_env, _now
trading/event_router.py	Project file.		build_event_router_report, event, write_report, read_json
trading/execution_ledger_split.py	Build split BUY/SELL intent ledgers and structured PnL reports.		build_execution_and_pnl_ledgers, buy_intents, protective_sell_intents, gap_escape_intents, profit_sell_intents, event_risk_sell_intents, _sell_plan_intents, options_plan_intents, persist_intents, build_pnl_ledger_report
trading/fast_order_guidance.py	Fast order guidance from scanner/news events. Guidance only; no order submission.		run
trading/gap_escape_manager.py	Project file.	GapEscapeConfig	load_gap_escape_config, build_gap_escape_report, gap_escape_row, gap_escape_blocked_reason, _trigger_reason, _active_stop_price, _active_stop_limit_price, _worst_loss_by_symbol, write_report, _bool_env
trading/gap_risk_manager.py	Project file.	GapRiskConfig	load_gap_risk_config, build_gap_risk_report, gap_risk_row, classify_symbol_shock, write_report, _long_positions, _latest_net_liquidation, _dedupe_shocks, _float, _float_env
trading/ibkr_callback_bridge.py	REAL_IBKR callback bridge into logical realtime data buses.	IbkrCallbackBridge	run_ibkr_callback_dry_run, stock_contract
trading/ibkr_callback_wiring_audit.py	Audit whether real IBKR callbacks are wired into RealtimeAccountStateBus.		build_ibkr_callback_wiring_audit, callback_row, bus_state_sources, callback_defined, callback_wires_bus, latest_bus_timestamp_for_callback, callback_notes, callback_wired, tws_socket_available, update_realtime_reports
trading/ibkr_news.py	IBKR news interface diagnostics.		run_interface_tests
trading/ibkr_readonly.py	Project file.	_QuoteState, IbkrReadOnlyQuoteClient, IbkrReadOnlyQuoteSource, ParallelIbkrReadOnlyQuoteSource	_chunks, _env_bool
trading/ibkr_scanner.py	IBKR scanner diagnostics and local discovery fallback.		run
trading/ibkr_streaming.py	Project file.	StreamingQuoteState, IbkrStreamingQuoteClient, IbkrStreamingQuoteSource	_clean_symbols, _log_event, streaming_blocked_reason
trading/market_data.py	Project file.	Quote, Bar, QuoteSource, InMemoryQuoteCache, RingBuffer, SymbolRingBuffers	quote_stream, _bucket_start, _float_or_none, _int_or_none

Path	Purpose	Classes	Functions
trading/market_data_line_manager.py	Project file.	MarketDataLineConfig, MarketDataLinePlan, MarketDataLineManager	_unique
trading/market_quote_crosscheck.py	Cross-validate market session expectations against received quotes.		build_market_quote_crosscheck_report, crosscheck_row, classify_cross_validation_result, summarize, write_report, _markdown, _read_json, _now
trading/market_session.py	US equity market-session classification and reporting. This module is intentionally independent from quote acquisition. A missing bid/ask can be a normal closed-session outcome, so	MarketSessionState, MarketSessionConfig, CalendarStatus, MarketSessionSnapshot, _ContractDetailsClient	config_from_env, current_market_session, write_market_session_report, fetch_ibkr_contract_details, market_session_md, classify_streaming_bid_ask_gap, parse_ibkr_trading_hours, _stock_contract, calendar_status, _state_for_time
trading/models.py	Project file.	TradeProposal, RiskDecision	
trading/module_interfaces.py	Project file.	StateFreshness, QuoteState, PnLState, ProtectionCoverageState, OpenOrderState, PositionState	
trading/news_candidate_router.py	Route structured news into candidate/risk events.		run
trading/news_event_monitor.py	News event monitor endpoint.		run
trading/news_event_structurer.py	Normalize raw news events into structured news events.		normalize
trading/news_provider_registry.py	Unified news provider registry for IBKR, RSS, SEC, earnings, cache, and fixtures.		build_registry
trading/options_hedge_planner.py	Project file.	OptionHedgeConfig	load_option_hedge_config, build_options_hedge_report, option_hedge_plan, write_report, _markdown, _bool_env, _float_env, _now
trading/order_intent_router.py	Order intent router. Emits synchronized intent events only.		route_synchronized_intents, write_conflict_report, run
trading/order_intents.py	Unified order intent model for Mode 9 paper automation ledgers.	OrderIntent	make_intent_id, utc_now
trading/order_rejections.py	Project file.		log_order_rejection, rejection_result, _parse_ib_error
trading/paper_audit_db.py	SQLite logging helpers for Mode 9 paper automation.		ensure_paper_automation_tables, insert_event, generic_decision_event, sqlite_table_summary, utc_now, _column_definitions
trading/paper_environment_audit.py	Project file.	PaperEnvironmentAuditItem, PaperEnvironmentAuditReport	build_environment_audit, write_environment_audit, main, fetch_health, _audit_items, _health_items, _item, _report_status, _next_steps, _settings_snapshot
trading/paper_execution_gate.py	Future paper execution gate. Report-only and default false.		run
trading/paper_order_executor.py	Future IBKR paper order executor interface. Disabled by default.		submit_order_if_enabled
trading/pool_builder.py	Dynamic six-layer pool builder.		run
trading/pool_manager.py	Project file.	PoolMembership	build_pool_manager_report, discovery_eligibility, architecture_report, write_reports, read_json
trading/pool_state.py	Canonical six-layer symbol pool state. This module keeps the six-layer pool logic separate from broker submission. It records why a symbol moves, but it does not place, cancel, or	PoolTransition, SymbolPoolState, PoolStateManager	quote_readiness, signal_failures, expired_signal, has_active_order, normalize_status, state_to_dict, state_from_dict, normalize_symbol, to_float, age_seconds
trading/position_guard.py	Project file.	PositionRecord, ProtectionStatus, ReadOnlyPositionsClient	read_positions, build_position_guard_report, write_report, evaluate_protection_status, _read_quotes, _read_streaming_quote_cache, _parse_timestamp, _float, _market_data_type_from_name, _env_bool
trading/position_lifecycle_manager.py	Position lifecycle manager for simulation, paper, and live-forbidden modes.	PositionLifecycle, PositionLifecycleManager	
trading/position_protection.py	Project file.	ProtectionRepairRecord, RepairPreviewProposal, StopConfig, RepairConfig, TestThenBulkSelection	run_position_protection, _repair_record_for_row, calculate_uncovered_qty, calculate_raw_stop_price, load_stop_config, load_repair_config, paper_operation_mode, build_repair_execution_plan, latest_test_repair_result, build_test_repair_result
trading/price_normalizer.py	Project file.	PriceNormalization	normalize_order_price, normalize_order_price_record, rounding_direction, log_normalization, _decimal_tick, _round_to_tick
trading/process_guard.py	Project file.	ProcessInfo, ExecutionLock	current_execution_processes, stop_duplicate_autonomous_processes, write_process_guard_report, append_history, read_lock, process_cwd, process_belongs_to_project, _inherited_owner, _parent_mode9_owner, _pid_alive
trading/profit_lock.py	Project file.		build_profit_lock_report, profit_lock_row, common_row, write_report, read_json, now
trading/realtime_account_state_bus.py	Near real-time account-centered state bus.	StateUpdate, RealtimeAccountStateBus	bootstrap_bus_from_reports, update_to_dict, read_json, utc_now
trading/realtime_account_sync.py	Realtime account sync orchestration for BUY/SELL synchronized decisions.		run_realtime_account_sync, sqlite_ledger_integrity_report, write_report
trading/risk.py	Project file.	RiskLimits, RiskEngine	
trading/scanner_universe_builder.py	Build scanner candidates for the discovery universe.		build_scanner_universe

Path	Purpose	Classes	Functions
trading/security_master.py	Project file.	SecurityMasterRecord	build_security_master_report, load_seed_records, load_security_master_records, record_from_universe_row, record_from_security_master_row, records_from_nasdaq_trader_rows, exchange_name, write_report
trading/sell_decision_engine.py	SELL decision engine. Emits SELL intents only; never calls IBKR directly.		build_sell_intent, priority, can_submit_order, submit_order
trading/service.py	Project file.	TradingService	
trading/session_calendar.py	Project file.	TradingSession, SessionSnapshot	classify_trading_session, _session_for_eastern, _as_utc
trading/session_policy.py	Project file.	SessionRiskPolicy, SessionOrderContext, PolicyDecision	default_session_policies, policy_for_session, validate_session_order, _policy, _bool_env, _int_env, _float_env
trading/simulated_order_executor.py	Local simulated order executor.		run
trading/simulated_pnl.py	Local simulated PnL helpers.		refresh
trading/simulated_position_ledger.py	Local simulated position ledger helpers.		refresh
trading/simulation.py	Project file.	StrategySimulationConfig	run_strategy_simulation, quote_source, default_prices, default_profiles, value_pool_record
trading/sqlite_event_store.py	Append-only SQLite event store with WAL enabled.	SQLiteEventStore	
trading/state_bootstrap_manager.py	Bootstrap buses from local cache, then keep REAL_IBKR subscriptions available.	StateBootstrapManager	source_timestamp, write_report, read_json
trading/state_freshness_policy.py	Freshness policy for simulation and future IBKR paper execution readiness.	FreshnessThresholds	evaluate_freshness, is_stale, write_report
trading/state_timestamp_reconciler.py	Timestamp reconciliation for cache, simulation, and REAL_IBKR events.	ReconciledField, TimestampReconciler	should_accept, parse_time
trading/strategy.py	Project file.	CandidateProfile, ValueFilterConfig, FilterResult, ValuePoolFilter, MovingAverageConfig, SignalDecision	_average_available, _lower_is_better, _higher_is_better
trading/strategy_modules.py	Project file.	ConservativeTrendConfig, ModulePosition, ModuleDecision, PortfolioRiskConfig, PortfolioRiskDecision, PortfolioRiskGate	_position_market_value, _is_regular_session
trading/strategy_signals.py	Strategy signal only architecture.	StrategySignal	create_strategy_signal, write_report, submit_order
trading/subscription_health.py	Always-listening subscription state reporting.		build_subscription_health_report, default_streams, stream
trading/take_profit.py	Project file.		build_take_profit_report, take_profit_row, write_report, read_json, now
trading/trailing_profit.py	Project file.		build_trailing_profit_report, trailing_profit_row, write_report, read_json, now
trading/tws_paper.py	Project file.	OrderConfirmation, TwsConnectionStatus, TwsPaperClient, TwsPaperBroker	normalize_us_stock_price
trading/universe.py	Project file.	UniverseEntry, UniverseSelectionConfig, UniverseSelection	load_universe, select_universe, default_universe_symbols, symbols_csv, _entry_from_row, _clean_symbol_set, _float_or_none, _int_or_none, _bool_or_true, _bool_or_none
trading/workflow.py	Project file.		attach_workflow_step, workflow_step, workflow_error_payload
scripts/approve_candidate_to_universe.py	Approve a hard-audited candidate report into the local universe CSV. This is the rule gate between LLM research and strategy modules. It refuses to write when Python hard audit rej		parse_args, main, read_rows, write_rows, upsert_candidate, row_from_report
scripts/audit_ibkr_level1_streaming.py	Read-only IBKR Level 1 streaming capability audit. This script never places, modifies, or cancels orders. It only requests contract details and Level 1 market data subscriptions, t	Level1AuditClient	main, _stock_contract
scripts/audit_paper_trading_readiness.py	Audit local readiness for a one-share IBKR paper trial.		
scripts/auto_paper_limit_from_quote.py	Create a paper limit order from the latest IBKR quote. This is the fast local execution harness for testing the full paper-trading path. It reads a quote, builds a limit proposal,		parse_args, main, latest_quote, order_payload, calculate_limit_price, quote_spread_bps, quote_payload, request_json, read_required, read_trade_session_token
scripts/benchmark_autonomous_agent_cycle.py	Benchmark autonomous-agent cycle timings across universe sizes. This uses the same Trading API health check and IBKR read-only quote source as the autonomous agent, but disables th		parse_args, main, summarize_size
scripts/benchmark_ibkr_quote_batches.py	Benchmark read-only IBKR quote request latency by batch size. This command only requests market data. It does not place orders.		parse_args, main, parse_symbols, parse_batch_sizes, summarize
scripts/build_account_reports.py	Build read-only account, position, open-order, and daily summary reports.		main, latest_pnl_row, flatten_open_orders, daily_summary, audit_requests_for_date, write_json, float_or_none
scripts/build_ibkr_callback_wiring_audit.py	Build IBKR callback wiring audit reports.		main
scripts/build_market_data_audit_report.py	Project file.		main, _read_json, _latest_pool_report, _code_hits, _execution_uses_delayed, _runtime_market_data_type, _symbols_with_valid_bid_ask, _symbols_with_stale_quotes, _symbols_with_no_permission

Path	Purpose	Classes	Functions
scripts/build_market_data_billing_safety.py	Audit market data billing safety for IBKR reqMktData calls.		main, build_report, audit_file, enclosing_function, arg_value, literal, symbol_source, market_data_type_nearby, action_taken, env_bool
scripts/build_market_data_line_report.py	Project file.		main, _read_json
scripts/build_mode9_execution_audit.py	Project file.		main, _read_json, _read_text, _bool_env
scripts/build_mode9_infrastructure_status.py	Build unified Mode 9 infrastructure status and audit reports.		main, build_all_reports, seed_current_sqlite_snapshots, ensure_sqlite_tables, build_sqlite_audit, build_simple_table_report, build_lot_protection_report, build_outside_rth_protection_report, build_fixed_symbol_audit, build_cycle_audit
scripts/build_paper_execution_gate.py	Project file.		
scripts/build_paper_pre_submit_review.py	Project file.		
scripts/build_paper_readiness_report.py	Build a manual paper readiness report from a validate-only plan.		
scripts/build_paper_session_runbook.py	Generate a paper-trial operator runbook from current reports.		
scripts/build_paper_session_status_page.py	Generate a static HTML paper-trial status page.		
scripts/build_paper_trial_evidence_bundle.py	Build a current-state evidence bundle for the paper-trial workflow.		
scripts/build_paper_trial_preflight_checklist.py	Build a one-share IBKR paper-trial preflight checklist.		
scripts/build_paper_trial_reconciliation.py	Build a post-paper-trial reconciliation report.		
scripts/build_paper_validation_plan.py	Build a validate-only paper trading plan from diagnostics JSON.		
scripts/build_pool_architecture_audit.py	Audit whether the six-layer pool architecture is implemented and wired.		main, build_reports, layer_report, membership_records, record, write_outputs, markdown, read_universe, read_json, sqlite_tables
scripts/build_position_guard_report.py	Project file.		main
scripts/build_project_completion_audit.py	Build the current project completion audit.		
scripts/build_quote_quality_report.py	Build a quote quality report from recorded IBKR quote samples.		
scripts/build_security_master.py	Build the persistent global security master. This is a data-prep tool only. It does not connect to TWS, request market data, or submit/cancel orders.		main, load_nasdaq_trader_url, load_nasdaq_trader_file, parse_pipe_text, write_records
scripts/build_strategy_catalog.py	Build the strategy module catalog.		
scripts/build_strategy_extension_status.py	Build report-only status for strategy extensions not allowed to trade yet.		build_status, latest_agent_report, write_reports, _bool_env, main
scripts/build_strategy_validation_plan.py	Project file.		
scripts/check_ibkr_quotes.py	Fetch read-only IBKR quotes and print the normalized cache snapshot. This command does not place orders. It is for checking what the current IBKR market-data permissions return for		parse_args, main
scripts/diagnose_market_data_channels.py	Diagnose read-only IBKR market data channels in order. The script only requests market data. It does not place orders.		parse_args, main
scripts/generate_daily_feishu_update.py	Generate a sanitized daily OpenClaw project update for Feishu.		parse_args, read_json, files_modified_since, universe_summary, latest_pnl, parse_timestamp, api_running, api_health_status, file_age_seconds, top_skip_reasons
scripts/generate_strategy_dashboard.py	Generate a local static strategy dashboard. The dashboard is a static HTML file with embedded simulation data. It does not connect to IBKR and does not place orders.		parse_args, main
scripts/hunt_paper_candidate.py	Hunt for a validate payload without paper order submission.		
scripts/list_tws_orders.py	List current TWS open orders and recent executions. This command is read-only. It does not place, modify, or cancel orders.	ReadOnlyOrdersClient	_number, parse_args, main, read_tws_orders
scripts/monitor_paper_trial_readiness.py	Safely monitor readiness for a one-share IBKR paper trial.		
scripts/record_account_pnl.py	Record read-only IBKR paper account P&L; samples over time. This script never places orders. It connects to TWS, requests account-level P&L; and account summary values, then appends	PnlSample, AccountPnlClient	parse_args, main, append_outputs, _float_or_none, _first_not_none, _sum_available
scripts/record_ibkr_quotes.py	Record read-only IBKR quote snapshots to CSV for research.		
scripts/refresh_paper_trial_reports.py	Refresh all paper-trial reports in safe order.		
scripts/refresh_position_protection_test_result.py	Refresh position protection test result from read-only TWS open-order output.		refresh_result, _find_order, main

Appendix

Glossary

Term	Meaning
Mode 9	Autonomous agent background mode described by project docs and reports.
Paper-only	Execution is limited to IBKR paper-trading environment; live mode is not validated.
cycle_id	Runtime cycle correlation key used across SQLite, reports, and review timelines.

Term	Meaning
snapshot_id	Immutable state snapshot reference for a decision cycle.
intent_id	Structured order intent identifier; not necessarily a submitted order.

Configuration Reference

- `TRADING\_MODE`: DRY_RUN or PAPER.
- `ALLOW\_TWS\_STAGING`, `ALLOW\_PAPER\_TRANSMIT`, `ALLOW\_OUTSIDE\_RTH`, `TRADING\_KILL\_SWITCH`: major safety switches.
- `TWS\_HOST`, `TWS\_PORT`, `TWS\_CLIENT\_ID`: TWS API connection values.
- `AUDIT\_DB`: defaults to `trading\_audit.sqlite3`.
- `MAX\_QUANTITY`, `MAX\_ORDER\_VALUE`, `MAX\_RISK\_PER\_ORDER`, `MAX\_DAILY\_NOTIONAL\_VALUE`: risk limits.

Facts Requiring Manual Confirmation

- Full system validation remains incomplete for this draft baseline.
- Some retention/cleanup policies are not encoded in source and are documented as not yet formalized.
- TWS desktop/API settings must be confirmed from the TWS UI during an operational run.
- Report-only modules marked NOT_STARTED or not connected in Mode 9 reports require owner prioritization.

Data Inventory

Path	Size	Rows	Columns
data/ibkr_quotes.csv	13934	90	sample, requested_at, symbol, timestamp, open, high, low, close
data/security_master.csv	2294513	13024	symbol, company_name, asset_type, primary_exchange, listing_exchange, currency, country, sector
data/us_equity_universe.csv	7762	71	symbol, name, exchange, sector, enabled, tags, average_volume, pe_ratio

Git/Version Information

Field	Value
Manual version	v0.1.0
Release name	Foundation Baseline
Chinese name	初始架构基线版
Git branch	feature/android-integration-mvp
Git commit	345fe379a8b06d1f7e1e1432edda1d78db54cb5f
Git remote	git@github.com:liuhaitian888-wq/openclaw-ibkr-paper-trading.git
Commit URL	https://github.com/liuhaitian888-wq/openclaw-ibkr-paper-trading/commit/345fe379a8b06d1f7e1e1432edda1d78db54cb5f
Dirty worktree at generation	yes